

# Leva.Framework.Storage

---

`Leva.Framework.Storage` is the provider-neutral persistence contract layer of ArchFramework. It defines repositories, journals, stored entries, versions, and common storage errors used by provider libraries and applications.

Storage exists so framework-owned data can be persisted without coupling the framework to EF Core, SQLite, files, or any other concrete persistence technology. It is intended for snapshots, runtime journals, small keyed records, and provider-neutral framework persistence. Application domain data with relationships and rich queries should usually use EF Core directly in the application storage layer.

`Leva.Framework.Storage` depends on `Leva.Framework.Core` so storage operations use the same `Result`, `Result<T>`, and `Error` values as the rest of the framework. Core must not depend on Storage, and Engine should not depend on Storage directly; hosts and infrastructure adapters should connect storage explicitly at the application boundary.

```
Leva.Framework.Storage
    -> Leva.Framework.Core
```

## Project overview

---

Storage is intentionally small. Repositories are the general durable value store. They store keyed values, return values with provider metadata through `StorageEntry<T>`, and use optional expected versions for optimistic concurrency. Framework snapshots do not need a separate snapshot-store contract because they can be stored through a repository with a string key.

Journals are append-only stores for durable history, audit trails, runtime records, or replayable entries. A journal appends records and reads them in provider-assigned version order, optionally after a known version and with a maximum count.

Storage is not a replacement for EF Core. EF Core should be used directly by applications for relational domain data, navigation properties, migrations, and complex queries. Storage should stay focused on simple provider-neutral persistence that framework libraries and hosts can swap between memory, files, SQLite, or future providers.

## Files and classes

---

### Repository contracts

`IRepository<TId, TValue>` - Stores keyed values without exposing provider-specific persistence details. It checks existence, loads one value, loads all values, saves with an optional expected version, and deletes with an optional expected version.

## Journal contracts

`IJournal<TEntry>` - Stores append-only entries for durable history, audit trails, runtime records, or replayable data. It appends one entry and reads entries in version order, optionally after a known version and with a maximum count.

## Shared storage values

`StorageEntry<T>` - Represents one stored value together with provider-owned metadata such as version, creation time, and update time. `StorageVersion` - Represents a positive provider-assigned storage revision or journal sequence number. `StorageErrors` - Creates common structured storage errors for missing records, version conflicts, unavailable providers, and failed operations.